

RESULTADOS DE TRATAMIENTO DE AUDIO

Experiencia SONIDO · CADI Percepción Computacional

Semanas 4, 5 y 6

Autor: Jerson Baquero Cháves

Audio analizado: archivo_prueba.m4a

Octubre de 2026

Código y evidencias en GitHub:

https://github.com/Jerson09/Informe_de_resultados_de_tratamiento_de_audio_Jerson

1. Introducción y problemática

En esta actividad se tomó una grabación de voz y se le hizo un "arreglo": se revisó cómo sonaba, se encontraron sus defectos y se limpió con un programa de computador. El objetivo es mostrar, paso a paso y con evidencias, qué se hizo y qué cambió.

Problema encontrado. Al revisar el audio se vio que tenía tres defectos:

- **Un golpe grave al comienzo** (en los primeros 0,7 segundos) que hizo que el sonido se "aplastara": 1082 puntos del audio llegaron al tope de volumen y se recortaron.
- **Ruido de fondo** (un "siseo" suave) que se escucha en las pausas entre palabras.
- **Sonidos que sobran:** tonos extremadamente graves y extremadamente agudos que no hacen parte de la voz.

Palabras que conviene conocer: Frecuencia = qué tan grave o agudo es un sonido (se mide en Hz; la voz hablada está entre unos 100 y 4000 Hz). Decibel (dB) = unidad para medir volumen; entre más negativo, más bajito. Filtro = herramienta que deja pasar unos tonos y quita otros. Normalizar = ajustar el volumen general sin deformar el sonido.

2. Descripción de la señal de audio

Característica	Dato
Fuente u origen	Archivo de prueba entregado en la actividad (archivo_prueba.m4a)
Formato	M4A (audio comprimido, similar al MP3)
Duración	11.71 segundos
Frecuencia de muestreo	48000 Hz (el audio guarda 48.000 "fotos" del sonido por segundo)
Canales	2 (estéreo), pero ambos suenan casi igual
Volumen máximo	1 (el tope; no puede pasar de 1)
Puntos recortados	1082

Lo más importante: es una grabación de voz con pausas entre palabras, con un golpe fuerte al inicio, y como ambos canales son casi iguales se puede tratar como un solo canal.

3. Captura u obtención del audio

No se grabó un audio nuevo: se usó el archivo de prueba que entregó la actividad (archivo_prueba.m4a). Ese archivo está guardado en el repositorio de GitHub del proyecto para que cualquiera pueda repetir el ejercicio.

4. Preprocesamiento

"Preprocesar" significa preparar el audio antes de analizarlo. Se hicieron estos pasos, en este orden:

Paso	Qué se hizo	Para qué sirve
1	Abrir el archivo y pasarlo a .wav	Python no lee .m4a directamente, así que se convierte primero.
2	Juntar los dos canales en uno	Ambos eran casi iguales; así se trabaja con una sola señal.
3	Centrar el audio en cero	El audio estaba "corrido" hacia abajo por el golpe inicial.
4	Quitar los tonos muy graves (menos de 80 Hz)	Elimina el golpe del comienzo.
5	Quitar los tonos muy agudos (más de 8000 Hz)	Casi todo lo que hay ahí es ruido, no voz.
6	Normalizar el volumen	Deja el sonido fuerte pero sin recortarse.

5. Desarrollo de la solución

Se usó el lenguaje Python con tres herramientas gratuitas: NumPy (hace las cuentas con los números del audio), SciPy (trae los filtros) y Matplotlib (dibuja las gráficas). También se usó FFmpeg, solo para convertir el .m4a a .wav.

El programa completo se llama `tratamiento_audio.py` y está comentado en español, explicando cada paso. Se encuentra en el repositorio:

https://github.com/Jerson09/Informe_de_resultados_de_tratamiento_de_audio_Jerson. Basta con ejecutarlo para obtener todos los resultados de este informe.

Evidencia de ejecución. La Figura 1 muestra el programa funcionando y los números que imprime al terminar.

```
$ python tratamiento_audio.py
{
  "frecuencia_muestreo_hz": 48000,
  "duracion_s": 11.71,
  "volumen_maximo": 1.0,
  "muestras_recortadas": 1082,
  "desplazamiento_continuo": -0.0193,
  "ruido_de_fondo_original_db": -47.1,
  "ruido_de_fondo_limpio_db": -54.2,
  "muestras_recortadas_limpio": 0,
  "porcentaje_silencio": 37.8,
  "frecuencia_mas_fuerte_hz": 393.0
}
```

Figura 1. Ejecución del programa y resultados que muestra.

6. Código relevante

Solo se muestran las partes clave. Los textos que empiezan con `#` son comentarios: explican el código y el computador los ignora.

6.1 Abrir el audio

```
subprocess.run(["ffmpeg", "-y", "-i", "archivo_prueba.m4a",  
               "resultados/audio_original.wav"]) # convertir a .wav  
fs, datos = wavfile.read("resultados/audio_original.wav")  
audio = datos.astype(float) / 32768 # valores entre -1 y 1  
audio = audio.mean(axis=1) # juntar los 2 canales en 1
```

6.2 Limpiar: quitar graves y agudos que sobran

```
limpio = audio - audio.mean() # centrar en cero  
quita_graves = signal.butter(4, 80, "highpass", fs=fs, output="sos")  
limpio = signal.sosfiltfilt(quita_graves, limpio) # quita < 80 Hz  
quita_agudos = signal.butter(6, 8000, "lowpass", fs=fs, output="sos")  
limpio = signal.sosfiltfilt(quita_agudos, limpio) # quita > 8000 Hz
```

6.3 Ajustar el volumen (normalizar)

```
# El punto más fuerte queda en 0.7: alto, pero sin llegar al tope (1.0)  
limpio = limpio / np.abs(limpio).max() * 0.7
```

6.4 Medir el ruido de fondo

```
# Se parte el audio en trozos de 25 milisegundos y se mide el volumen de cada uno.  
# El ruido de fondo es el volumen de los trozos más bajitos (el 10% más silencioso).  
ruido = np.percentile(niveles_por_trozos(limpio), 10)
```

7. Resultados obtenidos

7.1 Forma de onda

Muestra cuánto "sube y baja" el sonido en el tiempo: las partes altas son voz fuerte y las partes planas son pausas.

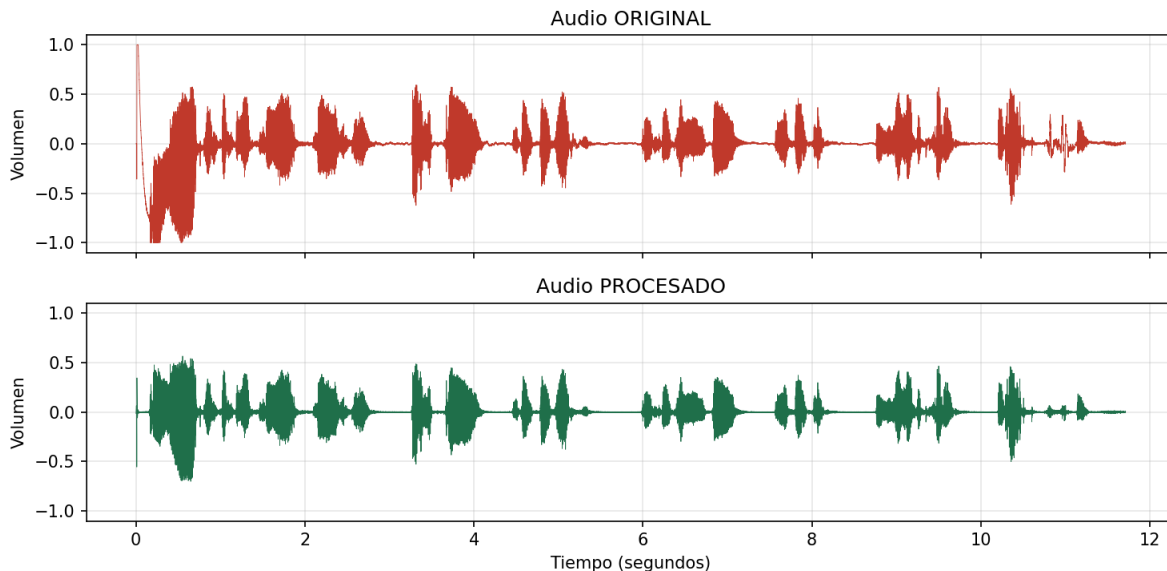


Figura 2. Audio original (arriba) y procesado (abajo).

7.2 Qué tonos tiene el audio

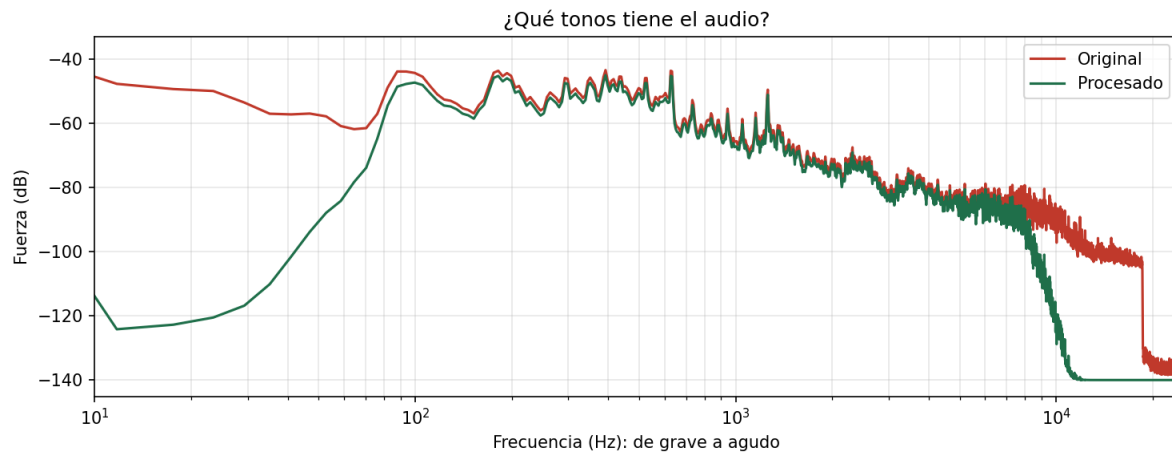


Figura 3. Fuerza de cada tono, de grave (izquierda) a agudo (derecha).

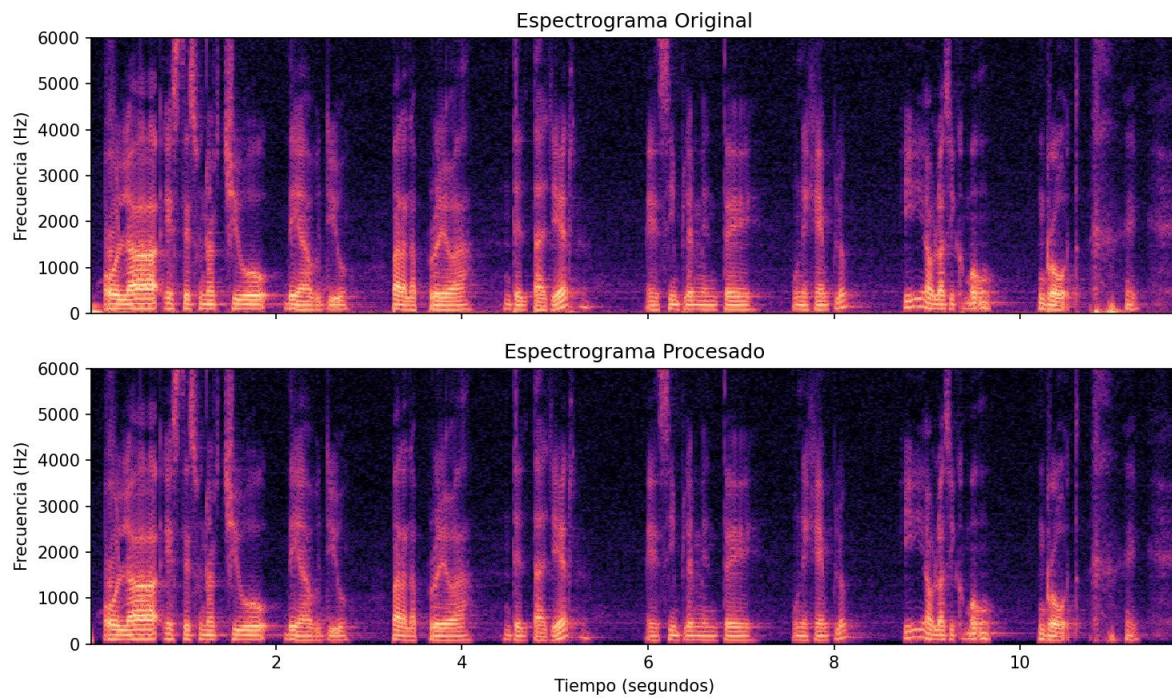


Figura 4. Espectrograma: tonos a lo largo del tiempo (más claro = más fuerte).

7.3 Números principales

Medida	Antes	Después
Puntos recortados (sonido aplastado)	1082	0
Desplazamiento (debería ser 0)	-0.0193	cerca de 0
Ruido de fondo	-47.1 dB	-54.2 dB
Porcentaje de silencio	—	37.8 %
Tono con más fuerza	—	393 Hz

8. Análisis e interpretación

Forma de onda (Figura 2). En el audio original, el comienzo se ve "colgado" hacia abajo y tocando el borde de la gráfica: es el golpe grave, y por eso hay puntos recortados. En el audio procesado esa parte ya no se sale del borde y todo queda centrado. Lo que se ve de la voz (los "montecitos") se conserva igual.

Tonos (Figura 3). A la izquierda de la gráfica (tonos muy graves) la línea roja está muy por encima de la verde: eso era el golpe, y el filtro lo eliminó. A la derecha (tonos muy agudos) pasa algo parecido con el ruido. En el centro, donde está la voz, las dos líneas casi se tapan: la voz no se dañó. La mayor parte de la fuerza está entre 100 y 1000 Hz, que es la zona de la voz hablada.

Espectrograma (Figura 4). Se parece a un "mapa de calor" del sonido. En el procesado desaparece el borrón de la franja más baja (el golpe) y de la parte más alta; la voz sigue viéndose en las mismas zonas.

Ruido de fondo. Bajó de -47.1 dB a -54.2 dB, unos 7 dB menos. Es una mejora real, pero modesta: los filtros quitan lo muy grave y lo muy agudo, pero no pueden quitar el ruido que está "mezclado" en los mismos tonos de la voz.

Conclusión. El tratamiento arregló el defecto más grave (el golpe y el recorte del inicio) y redujo un poco el ruido, sin dañar la voz. Lo que quedó recortado no se puede recuperar: la información que se perdió en el golpe ya no existe.

9. Dificultades encontradas

- **El formato M4A.** Python no lo abre directamente; se resolvió convirtiéndolo a .wav con FFmpeg.
- **Entender qué eran los números.** Al comienzo cuesta saber qué significan cosas como "frecuencia de muestreo" o "decibeles". Se resolvió dibujando gráficas y comparando antes y después.
- **Elegir hasta dónde filtrar.** Si el filtro es muy fuerte se daña la voz; si es muy suave, no quita el golpe. Se escogió 80 Hz y 8000 Hz porque la voz queda entre esos valores.
- **No poder recuperar lo recortado.** Se aceptó como una limitación del audio original.

10. Conclusiones

- Antes de arreglar un audio hay que revisarlo: medir su volumen, mirar su forma y sus tonos. Así se descubre qué está mal.
- Los filtros sirven para quitar sonidos que sobran (muy graves o muy agudos) sin dañar la voz.
- Las gráficas permiten comprobar con los ojos lo que el filtro hizo con el sonido.
- Un audio mal grabado no se arregla del todo: el tratamiento mejora, pero no recupera lo que ya se perdió.

11. Recomendaciones

- Al grabar: no tocar ni mover el teléfono al empezar, y dejar un par de segundos de silencio al principio.
- Hablar a una distancia y volumen adecuados para que el sonido no llegue al tope y se recorte.
- Para quitar más ruido de fondo se puede probar una técnica de "reducción de ruido" más avanzada.
- Escuchar el audio original y el procesado, uno tras otro, para confirmar con el oído que el resultado suena mejor.
- Este mismo proceso sirve para limpiar entrevistas, clases grabadas o audios que luego se quieran transcribir a texto.

Código completo y evidencias:

[https://github.com/Jerson09/Informe de resultados de tratamiento de audio Jerson](https://github.com/Jerson09/Informe_de_resultados_de_tratamiento_de_audio_Jerson)